Practical No.: 03

NAME: Bhalerao Tanvi Vitthal
ROLLNO.: 08
COURSE: AI&DS
CLASS: BE
SUB: Computer Laboratory-IV
Title: Recurrent Neural Networks (RNN) for Time Series Prediction Using LSTM/GRU.

0.1 Import Required Libraries

[17]:
```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.sequence import pad_sequences

import matplotlib.pyplot as plt
import numpy as np
```

0.2 Load Dataset

[18]:
```
# Load IMDB dataset (top 10,000 most frequent words)
vocab_size = 10000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
```

```
Training samples: 25000
Testing samples: 25000
```

0.3 Data Preprocessing

[19]:
```
# Pad sequences to same length
max_length = 200

x_train = pad_sequences(x_train, maxlen=max_length)
x_test = pad_sequences(x_test, maxlen=max_length)
```

0.4 Build LSTM Model

[20]:
```
model = Sequential()

model.add(Embedding(input_dim=vocab_size, output_dim=128,␣
 ↳input_length=max_length))
model.add(LSTM(128, return_sequences=False))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

```
D:\Old ssd\Anaconda3\Anaconda\Lib\site-
packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

0.5 Compile the Model

[21]:
```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

0.6 Train the Model

[22]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/10
313/313                 81s 247ms/step -
accuracy: 0.7850 - loss: 0.4553 - val_accuracy: 0.8608 - val_loss: 0.3294
Epoch 2/10
313/313                 87s 261ms/step -
accuracy: 0.8919 - loss: 0.2754 - val_accuracy: 0.8620 - val_loss: 0.3611
Epoch 3/10
313/313                 77s 245ms/step -
accuracy: 0.9238 - loss: 0.2009 - val_accuracy: 0.8678 - val_loss: 0.3400
Epoch 4/10
313/313                 77s 245ms/step -
accuracy: 0.9449 - loss: 0.1531 - val_accuracy: 0.8570 - val_loss: 0.3799
Epoch 5/10
313/313                 77s 247ms/step -
accuracy: 0.9599 - loss: 0.1159 - val_accuracy: 0.8578 - val_loss: 0.4451
Epoch 6/10
313/313                 78s 248ms/step -
accuracy: 0.9706 - loss: 0.0856 - val_accuracy: 0.8640 - val_loss: 0.5212
Epoch 7/10
313/313                 78s 249ms/step -
accuracy: 0.9704 - loss: 0.0856 - val_accuracy: 0.8620 - val_loss: 0.4995
Epoch 8/10
313/313                 77s 246ms/step -
accuracy: 0.9822 - loss: 0.0583 - val_accuracy: 0.8606 - val_loss: 0.5834
Epoch 9/10
313/313                 78s 249ms/step -
accuracy: 0.9863 - loss: 0.0431 - val_accuracy: 0.8546 - val_loss: 0.6785
Epoch 10/10
313/313                 77s 247ms/step -
accuracy: 0.9866 - loss: 0.0432 - val_accuracy: 0.8428 - val_loss: 0.7031
```

0.7 Evaluate the Model

[23]:
```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_accuracy)
```

```
782/782                 44s 57ms/step -
accuracy: 0.8425 - loss: 0.7108
Test Accuracy: 0.8424800038337708
```

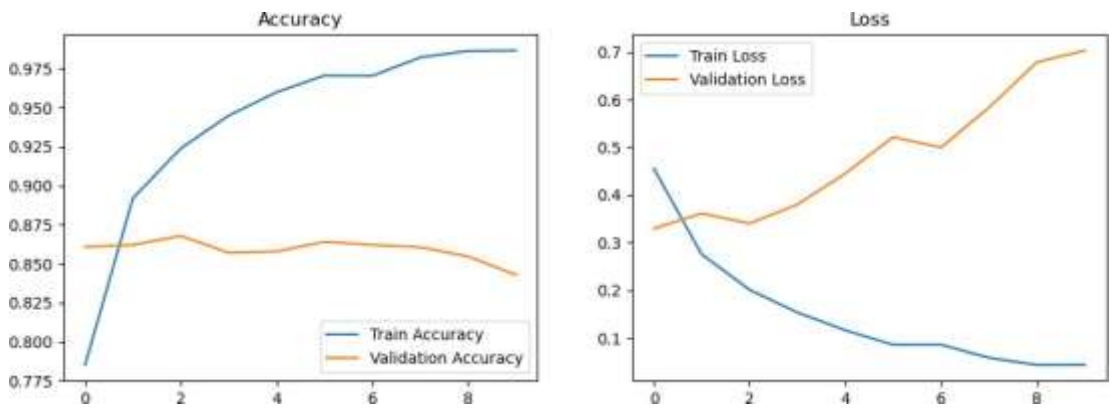
0.8 Plot Accuracy and Loss

[24]:
```
plt.figure(figsize=(12,4))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy")

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss")

plt.show()
```



0.9 Predict Sentiment for New Review

[25]:
```
# Example review (encoded manually for demo)
sample_review = x_test[0].reshape(1, -1)
prediction = model.predict(sample_review)

if prediction > 0.3:
    print("Positive Review")
else:
    print("Negative Review")
```

```
1/1                 0s 310ms/step
Positive Review
```